

Formal Specification of Linux eBPF Instruction Set Architecture in Sail

Yazhou Tang¹, Shenghao Yuan^{1,†}, Jean-Pierre Talpin², Mingshuai Chen^{1,†}

¹Zhejiang University, China ²INRIA, France

Presenter: Yazhou Tang

eBPF Workshop 2026 · Prague, Czech Republic · September 29, 2026

Section 1

Introduction

eBPF standardization



¹Thaler. 2024. BPF Instruction Set Architecture (ISA). RFC 9669. (Oct. 2024).
doi:10.17487/RFC9669

Formal specification?



Do we need a formal model/specification?

Formal specification?



Do we need a formal model/specification? **For standardization: may not.**

Formal specification?



Do we need a formal model/specification? **For standardization: may not.**

In the IETF discussion on 2023²

- Watson Ladd: “the specification seems to use an underspecified C pseudo code for operations vs **defining them mathematically.**”
- Will Hawkins: “perhaps **Sail** is a way to proceed.”
- Alexei Starovoitov: “imo Sail is too researchy to have practical use ...**I don’t see an immediate benefit to standardization.** There could be other use cases, no doubt, but standardization is our goal.”

²<https://mailarchive.ietf.org/arch/msg/bpf/dDzJ-CYLMDxdi2hQamYDfPWJt0g/>

Formal specification?



Do we need a formal model/specification?

Formal specification?



Do we need a formal model/specification? **For research: YES!**

Formal specification?



Do we need a formal model/specification? **For research: YES!**

① Alan Jowett discussed in the *BPF Conformance* talk on bpfconf'24.³

(How to) Generating new tests

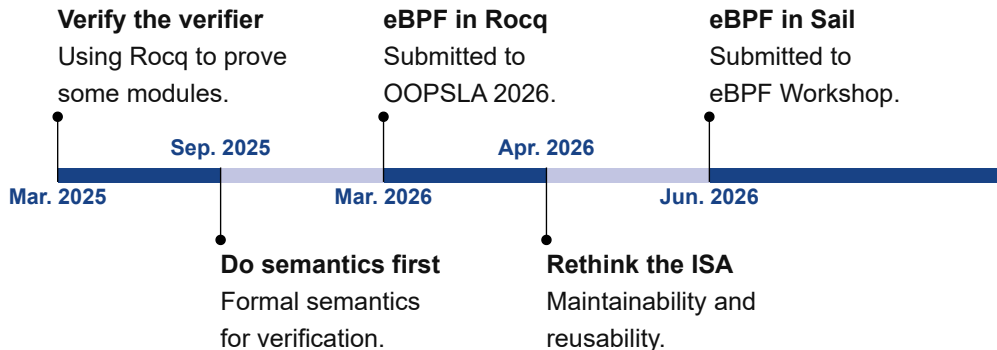
- Manually reviewing the BPF ISA specification – **Time consuming and error prone.**
- Generate from **machine readable model** of the BPF ISA specification.

³Source: github.com/Alan-Jowett/bpf_conformance

Our journey to Sail



② Formal foundation for verification research. (from our perspective)



Our OOPSLA work: eBPF in Rocq



浙江大学
ZHEJIANG UNIVERSITY



1. Mechanization

153 sequential instructions

Executable model
CompCert memory

2. Validation

1,700+ retained Linux selftests

Per-step register state comparison

3. Application

TNum soundness proofs (32 / 64 bits)

Upstream verifier fixes and improvements

A foundation for verifier proofs in Rocq⁴.

Next: make the ISA itself easier to maintain and reuse.

⁴Yuan et al. 2026. Formalizing the Linux eBPF core ISA: a mechanized operational semantics and its real-world applications. *Proc. ACM Program. Lang.*, 10, OOPSLA2, (Oct. 2026)

So the reasons of this work...



Why formal specification?

- ① Formal language vs natural language: more precise.
- ② Use formal language as a foundation of research.

So the reasons of this work...



Why formal specification?

- 1 Formal language vs natural language: more precise.
- 2 Use formal language as a foundation of research.

Why use Sail?

Sail⁵ is more readable than Rocq (or other prover).

Semantics in Rocq

- Functional-style
- handwritten decoder
- handwritten spec

Semantics in Sail

- Imperative-style
- auto-generated decoder
- auto-generated spec

⁵Source: github.com/remis-project/sail

Why use Sail? (cont.)

There are many semantics/specification DSLs.

ISA	Formal language	Status
Intel x86	Intel ISA Specification Language	Future official source (2026 preview)
Arm	ASL	Official specification language (2017)
RISC-V	Sail	Official formal model (2019)
WebAssembly	SpecTec	Official standard's source (2025)

Why use Sail? (cont.)



There are many semantics/specification DSLs.

ISA	Formal language	Status
Intel x86	Intel ISA Specification Language	Future official source (2026 preview)
Arm	ASL	Official specification language (2017)
RISC-V	Sail	Official formal model (2019)
WebAssembly	SpecTec	Official standard's source (2025)

Sail vs SpecTec

- ☒ Designed specially for ISA semantics: encoding, bitvectors, imperative, ...
- ☒ Stronger downstream ISA ecosystem: emulator, prover, concurrency analysis, ...
- ☒ Structured natural-language prose (SpecTec has)

Section 2

Model

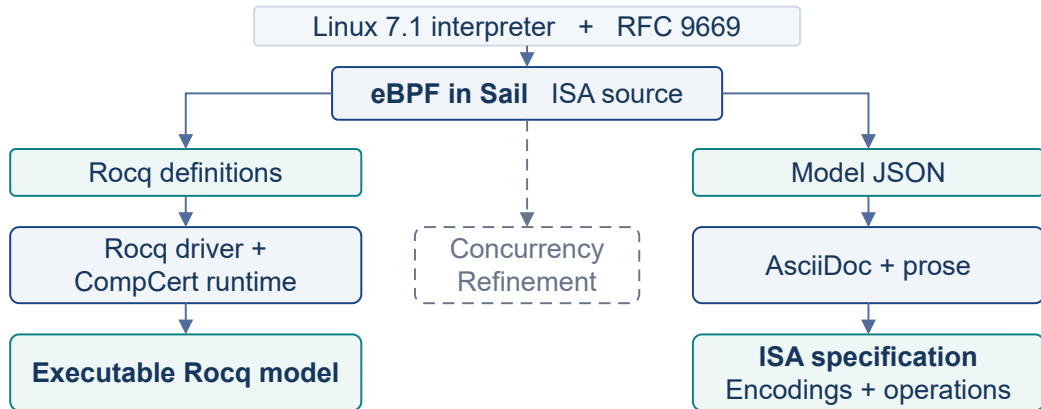
One Sail model, multiple artifacts



■ Source / handwritten

■ Generated

□ Future



Sail example: Syntax



BPF_SDIV32: signed 32-bit division

1 / 3

Sail instruction type

```
1 enum src_kind = { SRC_K, SRC_X }
2 newtype regidx = RegIdx : range(0, 10)
3 type imm32 = bits(32)
4
5 union clause instruction =
6   BPF_SDIV32 : (src_kind, regidx, regidx, imm32)
```

- SRC_K selects an immediate; SRC_X selects a register.
- Types capture register bounds and bit widths.

Sail example: Encoding



BPF_SDIV32: immediate-source form

2 / 3

Sail bidirectional mapping

```

1 mapping clause encdec =
2   BPF_SDIV32(SRC_K, dst, RegIdx(0), imm)
3   <-> imm @ 0x0001
4       @ encdec_reg(RegIdx(0)) @ encdec_reg(dst)
5       @ encdec_alu_code(DIV_OR_SDIV)
6       @ encdec_src_kind(SRC_K) @ encdec_alu_class(ALU32)

```

- 64-bit word: imm[32] | off[16] | src[4] | dst[4] | opcode[8].
- @ concatenates fields; <-> relates the AST and bits.
- off = 0x0001 selects signed division.

Sail example: Encoding (cont.)



BPF_SDIV32: register-source form

2 / 3

Sail bidirectional mapping

```
1 mapping clause encdec =  
2   BPF_SDIV32(SRC_X, dst, src, imm)  
3   <-> imm @ 0x0001  
4       @ encdec_reg(src) @ encdec_reg(dst)  
5       @ encdec_alu_code(DIV_OR_SDIV)  
6       @ encdec_src_kind(SRC_X) @ encdec_alu_class(ALU32)
```

- 64-bit word: imm[32] | off[16] | src[4] | dst[4] | opcode[8].
- @ concatenates fields; <-> relates the AST and bits.
- off = 0x0001 selects signed division.

Sail example: Execution



BPF_SDIV32: Sail execution (3 / 3)

```

1 function alu32_sdiv(dst : bits(64), src : bits(64)) -> bits(32) = {
2   let d_s = signed(dst[31 .. 0]);
3   let s_s = signed(src[31 .. 0]);
4   if s_s == 0 then { 0x0000_0000 }
5   else if (s_s == -1) & (d_s == alu32_signed_min()) then {
6     to_32bits(alu32_signed_min())
7   } else { to_32bits(tdiv_int(d_s, s_s)) }
8 }
9 function clause execute BPF_SDIV32(srck, dst, src, imm) =
10   finish_alu32(dst, alu32_sdiv(X(dst), alu32_src(srck, src, imm)))

```

- tdiv_int: truncation toward zero; signed minimum is -2^{31} .
- finish_alu32: zero-extend the result into dst and increment PC.

What does the current model cover?



Linux 7.1 interpreter

181 instruction forms

153 sequential

28 atomic

Executable sequential semantics

- ALU, memory, branches
- Calls, returns, tail calls
- Instruction decoding and state updates

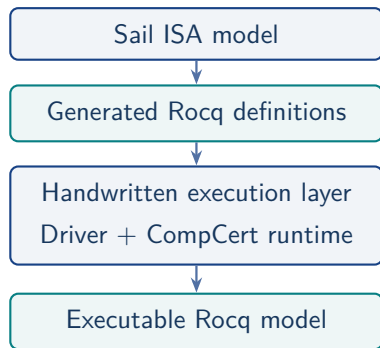
Atomic instruction interfaces

- Encodings and register effects
- Memory-effect annotations
- **No concurrency model**

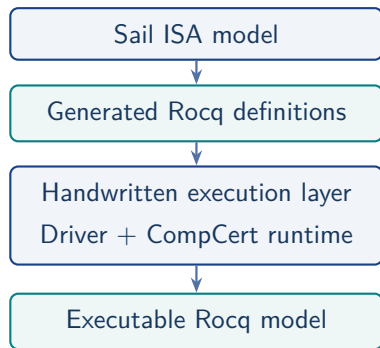
Notes:

- ① State includes registers, call frames, etc.; memory is maintained by backend.
- ② does not include the full helper / kfunc environment.

Keeping the connection to Verification



Keeping the connection to Verification



The memory bridge

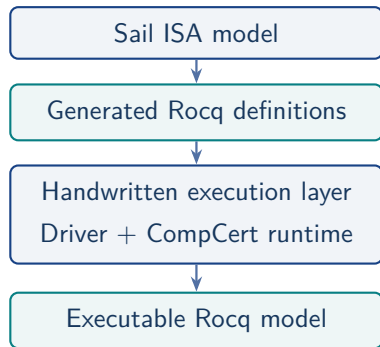
Sail: 64-bit concrete addresses and abstract memory effects.

Runtime: map address ranges to CompCert blocks.

Diagram illustrating the memory bridge mapping:

$$\begin{array}{c} \text{address} + \text{size} \\ \downarrow \\ \text{block} + \text{offset} \\ \downarrow \\ \text{Mem.load} / \text{Mem.store} \end{array}$$

Keeping the connection to Verification



The memory bridge

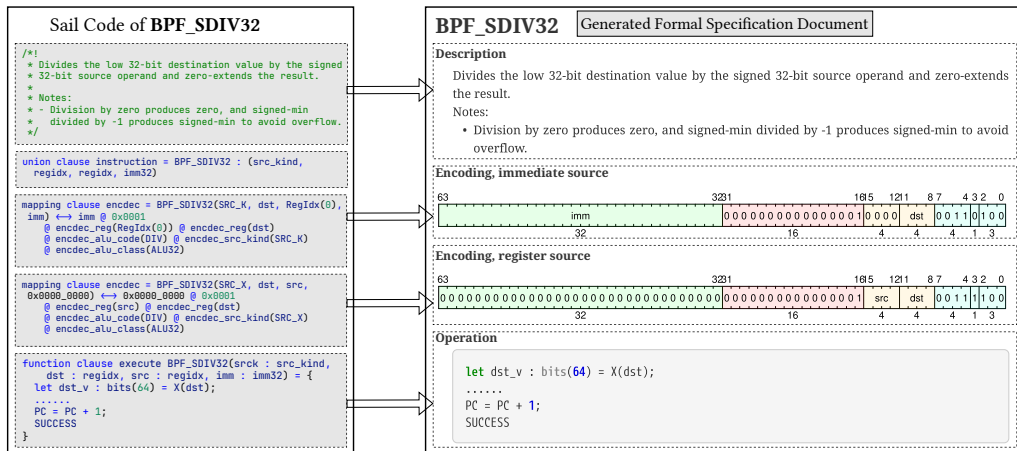
Sail: 64-bit concrete addresses and abstract memory effects.

Runtime: map address ranges to CompCert blocks.

address + size
↓
block + offset
↓
Mem.load / Mem.store

Sail is the ISA representation layer; Rocq remains the verification layer.

From Sail to an ISA specification



From Sail to an ISA specification (cont.)



Handwritten AsciiDoc framework

```
1 === BPF_SDIV32
2
3 Description::
4 +
5 --
6 include::sailcomment:execute[clause="BPF_SDIV32(_, _, _, _)"]
7 --
8
9 Encoding, immediate source::
10 include::sailwavedrom:encdec[left-clause="BPF_SDIV32(SRC_K,_,_,_)",type=mapping,right]
11
12 Encoding, register source::
13 include::sailwavedrom:encdec[left-clause="BPF_SDIV32(SRC_X,_,_,_)",type=mapping,right]
14
15 Operation::
16 +
17 sail::execute[clause="BPF_SDIV32(_, _, _, _)",part=body,unindent]
```

Section 3

Discussions

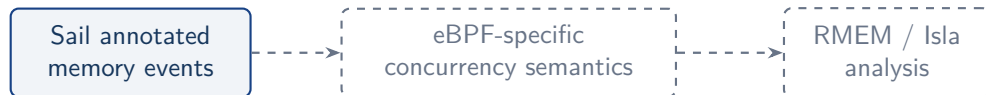
Atomic effects and concurrency



We represent the memory effects, and left the backends to handle them.

- Variety: what kind of memory action?
- Strength: what ordering strength?

Access	Variety	Strength
Plain load / store	AV_plain	AS_normal
Atomic read-modify-write	AV_atomic_rmw	AS_normal
Load-acquire / store-release	AV_plain	AS_rel_or_acq



Source: model/mem/mem_interface.sail. Dashed components are future work.

Atomic example



BPF_ATOMIC_ADD_W

```
1 let addr : bits(64) = to_64bits(unsigned(X(dst)) + signed(off));
2 let old : bits(64) = zero_extend(
3   ebpf_read_mem_bytes(Read_atomic_rmw, addr, 4)
4 );
5 let src_v : bits(64) = X(src);
6 let replacement : bits(64) = zero_extend(to_32bits(
7   unsigned(old[31 .. 0]) + unsigned(src_v[31 .. 0])
8 ));
9 let stored : bool =
10  ebpf_write_mem_bytes(Write_atomic_rmw, addr, 4, replacement[31 .. 0]);
11 if stored then {
12   PC = PC + 1;
13   SUCCESS
14 } else Illegal_Instruction("atomic store failed")
```

What Is Established Today?



We have to admit that this is an **early-stage** work.

Established so far

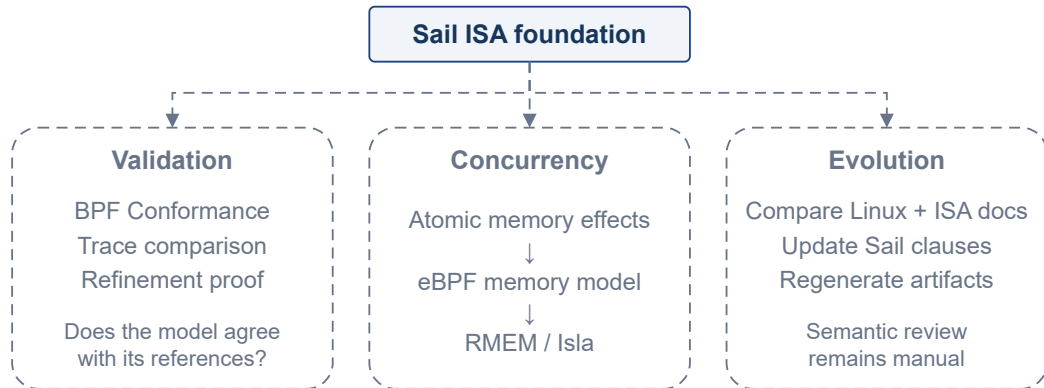
- Sail type checking
- Executable semantics with a Rocq runtime
- Manual cross-check against Linux 7.1 and RFC 9669

Not established yet

- Systematic validation against conformance tests and Linux
- Refinement proof to the earlier Rocq model

Next: **BPF Conformance** + **Differential testing** + **Rocq refinement**

Where Next?



Takeaways



浙江大学
ZHEJIANG UNIVERSITY

inria

① A Sail formalization of the Linux eBPF ISA

153 sequential instructions; interfaces for 28 atomic instructions.

② One ISA model, multiple artifacts

Executable semantics, Rocq definitions, ISA-style documentation.

③ A foundation for the next steps

Validation, concurrency, and maintenance across Linux releases.

Model + specification

github.com/OpenSourceVerif/sail-ebpf/tree/ebpf26-artifact

Thanks for listening!



浙江大學
ZHEJIANG UNIVERSITY

| *Inria*

Thank You

Yazhou Tang, Shenghao Yuan, Jean-Pierre Talpin, Mingshuai Chen

Modeled Linux eBPF instructions



Type	Instructions	Count
ALU32/ALU64	K/X: ADD, SUB, MUL, DIV, SDIV, OR, AND, LSH, RSH, MOD, SMOD, XOR, MOV, ARSH; NEG, MOVSX, LE/BE, BSWAP	72
Jumps/calls	JMP/JMP32 K/X: JEQ, JNE, JGT, JLT, JGE, JLE, JSET, JSGT, JSLT, JSGE, JSLE; JA, helper/kfunc/local calls, tail call, EXIT	51
Memory	LDX_MEM_B/H/W/DW, ST_MEM_B/H/W/DW, STX_MEM_B/H/W/DW, LDX_MEMSX_B/H/W	15
Probe loads	LDX_PROBE_MEM_B/H/W/DW, LDX_PROBE_MEMSX_B/H/W	7
Wide immediates	LD_IMM64, LD_MAP_FD, LD_MAP_VALUE_FD, LD_BTFF_ID, LD_FUNC, LD_MAP_IDX, LD_MAP_VALUE_IDX	7
Barrier	ST_NOSPEC	1
Sequential	Executable sequential semantics	153
Atomic RMW	W/DW: ADD, OR, AND, XOR, their FETCH variants; XCHG, CMPXCHG	20
Acquire loads	LOAD_ACQ_B/H/W/DW	4
Release stores	STORE_REL_B/H/W/DW	4
Atomic family	Instruction interfaces; concurrency model is future work	28
Total	153 sequential + 28 atomic instruction forms	181

Linux 7.1 baseline. K/X: immediate/register; B/H/W/DW: 8/16/32/64 bits.

Counts distinguish instruction forms, including widths and variants.

RMEM: Can an Atomic Update Be Lost?



Two atomic increments

Shared 32-bit counter: $x = 0$.

Both threads: BPF_ATOMIC_ADD_W

Same address $\&x$; source value 1.

Thread 1

Thread 2

atomic $x += 1$ atomic $x += 1$

Can the final x be 1?

Both succeed; no other writes.

Proposed check with RMEM⁶

- 1 Use Sail's read, add, and write steps for each instruction.
- 2 Explore **executions under rules** that preserve each atomic update.
- 3 Check that both completed updates give $x = 2$, never $x = 1$.

Key rule: the read and write belong to one atomic update.

⁶RMEM source: github.com/rem-s-project/rmem



Isla: Can the Same Bad Result Exist?

Two atomic increments

Shared 32-bit counter: $x = 0$.

Both threads: BPF_ATOMIC_ADD_W

Same address $\&x$; source value 1.

Thread 1

Thread 2

atomic $x += 1$ atomic $x += 1$

Can the final x be 1?

Both succeed; no other writes.

Proposed check with Isla⁷

- 1 Represent each read value as an unknown; each write is that value plus one.
- 2 Add **atomicity rules** and request a final state with $x = 1$.
- 3 Ask a SMT solver: a correct model should make these constraints unsatisfiable.

Bad candidate: both read 0, both write 1. Atomicity must rule this out.

⁷Isla source: github.com/rem-s-project/isla

Handwritten vs. Generated



Component	Origin
Sail ISA model	Handwritten
Rocq definitions	Generated
Rocq driver / CompCert adapter	Handwritten
Documentation JSON	Generated
Descriptive prose / AsciiDoc	Handwritten
Encoding / operation sections	Generated
Assembled PDF specification	Generated

Review and trust remain

ISA transcription, Sail tooling, and handwritten execution support.

Documentation detail

- `model/`: source manual.
- Polished manual: `model_manual/`, with helpers expanded by hand.
- Possible next step: automatic expansion.

Generated Rocq code does not automatically prove the ISA model correct.

A Known Shift Divergence



For RFC/interpreter/our Sail model

Mask register-based shift amounts:

32-bit: `shift & 0x1f`

64-bit: `shift & 0x3f`

e.g., a 32-bit shift by 32 becomes a shift by 0.

For JIT backends

Lower directly to native shifts for performance.

- Out-of-range behavior depends on the target architecture.
- Some JIT backends (e.g. PowerPC) would set result to 0.

A known specification / implementation divergence.

Paper, Discussion: A Shift-Semantics Divergence Case Study. Sail follows RFC 9669 masked semantics.